



We are developing an FYP titled:

“TASK-ORIENTED DEEP JOINT SOURCE-CHANNEL CODING FOR UAV NAVIGATION UNDER NOISY AND FADING WIRELESS CHANNELS”

The project is simulation-only and uses Python/PyTorch. We have already researched and verified the following areas:

1. DeepJSCC foundation and encoder/decoder structure.
2. Complex-to-real PyTorch representation.
3. Power normalization.
4. AWGN and Rayleigh fading channel models.
5. SNR/noise conventions.
6. CSI/equalization considerations.
7. Bandwidth ratio.
8. Conventional digital communication baseline.
9. Training/validation/test protocol.
10. Seeds, SNR ranges, bandwidth ratios, ablations, and statistical evaluation.
11. UAV mathematical model at a basic level.

Our UAV model is:

$$\mathbf{s}_t = [x_t, y_t, v_{x,t}, v_{y,t}, x_g, y_g]$$

with dynamics:

$$\mathbf{p}_{t+1} = \mathbf{p}_t + \mathbf{v}_t \Delta t$$

$$\mathbf{v}_{t+1} = \mathbf{v}_t + \mathbf{u}_t \Delta t$$

where $\mathbf{u}_t$ is bounded acceleration/control input.

The UAV communicates its state to a ground station. The communication system is:

$$\mathbf{s}_t \rightarrow f_\theta \rightarrow \mathbf{z}_t \rightarrow \text{wireless channel} \rightarrow \hat{\mathbf{z}}_t \rightarrow g_\phi \rightarrow \mathbf{r}_t \rightarrow \text{navigation controller} \rightarrow \mathbf{u}_t \rightarrow \mathbf{s}_{t+1}.$$

We need to determine the mathematically correct task-oriented training formulation before implementation.

IMPORTANT:

- Use ORIGINAL PAPERS whenever possible.
- Prioritize the original Diao 2024/2025 papers, Diao's PhD thesis, and other primary task-oriented communication/control papers.
- Do NOT invent equations, controller gains, thresholds, loss weights, training procedures, or architecture details.
- If something is not specified in the literature, explicitly write:
“NOT SPECIFIED IN SOURCE — PROJECT-SPECIFIC DECISION.”
- Clearly separate:
 1. Literature-supported facts
 2. Mathematical interpretation/inference
 3. Project-specific decisions
- Do not silently transfer CARLA/autonomous-driving parameters into the UAV problem.
- The goal is to produce a defensible mathematical formulation for our UAV project, not to claim that the literature already specifies our exact UAV system.

PART 1 — Verify the task-oriented objective in Diao's work

Study the relevant Diao papers and thesis and answer:

A. What exactly is the task variable?

Determine whether the task-oriented communication system is optimizing:

- reconstructed state,
- task representation,
- control action,
- future trajectory,
- task performance,
- or a combination.

Give the exact mathematical formulation from the source where available.

B. What does the decoder actually output?

Determine whether the decoder produces:

- reconstructed source/state,
- latent task representation,
- control command,
- future trajectory,
- or another representation.

Give the exact architecture/objective relationship.

C. What is the task loss?

Find the exact task-oriented loss used by Diao.

For example, determine whether it contains terms corresponding to:

$$\mathcal{L}_{\text{task}}$$

$$\mathcal{L}_{\text{control}}$$

$$\mathcal{L}_{\text{trajectory}}$$

information bottleneck terms,

or other components.

Give equations exactly or faithfully paraphrased from the source.

Explain what each term means.

D. DTCP / trajectory-guided control prediction

For the 2025 communication-computing-control work, verify:

- what DTCP means,
- what it predicts,
- whether it predicts control actions or trajectories,
- whether it is learned,
- how it interacts with communication,
- how its loss is defined,
- whether the controller is fixed or learned,
- whether communication and control are jointly optimized.

Do NOT assume the CARLA implementation directly transfers to UAV dynamics.

PART 2 — Determine the appropriate UAV task representation

For our UAV state:

$$\mathbf{s}_t = [x_t, y_t, v_{x,t}, v_{y,t}, x_g, y_g]$$

investigate which task representation is mathematically appropriate.

Consider these possibilities:

Option A — Full state reconstruction

Decoder outputs:

$$\hat{\mathbf{s}}_t$$

$$[\hat{x}_t, \hat{y}_t, \hat{v}_{x,t}, \hat{v}_{y,t}, \hat{x}_g, \hat{y}_g]$$

and

and a fixed controller uses $\hat{\mathbf{s}}_t$.

Option B — Goal-relative state

Define:

$$\mathbf{e}_p = \mathbf{p}_g - \mathbf{p}_t$$

and potentially:

$$\mathbf{e}_v = -\mathbf{v}_t$$

so the decoder outputs something such as:

$$\mathbf{r}_t = [e_x, e_y, v_x, v_y]$$

or another task-relevant representation.

Option C — Direct control prediction

Decoder directly produces:

$$\hat{\mathbf{u}}_t$$

which is then applied to the UAV.

Option D — Future trajectory / control sequence

Decoder predicts:

$$\hat{\mathbf{p}}_{t+1:t+H}$$

or

$$\hat{\mathbf{u}}_{t:t+H-1}.$$

For each option:

- find literature precedent if available,
- explain what objective it naturally supports,

- explain whether it is differentiable,
- explain whether it fits our project scope,
- identify advantages/disadvantages,
- and explicitly state if the literature does NOT provide enough evidence to choose one.

Do NOT simply choose the most sophisticated option.

PART 3 — Verify the UAV controller formulation

Investigate whether a simple fixed goal-reaching controller can be used to isolate the communication problem.

One candidate formulation is:

$$e_p = p_g - p_t$$

$$e_v = -v_t$$

$$u_t^* = K_p e_p + K_d e_v$$

followed by control saturation:

$$u_t = \text{clip}(u_t^*, -u_{\max}, u_{\max}).$$

Determine:

1. Whether this is mathematically appropriate for the stated double-integrator UAV model.
2. Whether a PD controller is sufficient for this project.
3. Whether literature provides suitable gain-selection guidance.
4. Whether K_p, K_d should be fixed.
5. Whether the controller should be learned jointly with the communication model.
6. Whether fixing the controller gives a cleaner evaluation of the communication system.

IMPORTANT:

Do NOT invent numerical values for:

- K_p
- K_d
- $u_{\max}$
- $v_{\max}$
- Δt

If the literature does not specify them for our exact model, mark them as:

PROJECT-SPECIFIC PARAMETERS — TO BE SELECTED AND JUSTIFIED SEPARATELY.

PART 4 — Most important: differentiable training objective

Our main problem is that the final navigation success criterion is:

$$\|p_T - p_g\| \leq r_g.$$

This is useful for evaluation, but the binary indicator

$$\mathbf{1}[\|p_T - p_g\| \leq r_g]$$

is not an appropriate direct gradient-training objective.

Investigate how task-oriented communication can instead be trained using differentiable surrogate losses.

Compare the following possible formulations.

Candidate 1 — Control imitation loss

If an ideal controller generates:

$$u_t^*$$

then train the communication system so that the received representation produces:

$$\hat{u}_t \approx u_t^*.$$

Possible loss:

\$\$
$$\frac{1}{T} \sum_{t=1}^T \|\hat{u}_t - u_t^*\|^2.$$

\$\$

Determine whether this formulation is supported by literature or would be our project-specific adaptation.

Candidate 2 — One-step state prediction/navigation loss

Use the communication output to generate a control input and predict:

$$\hat{s}_{t+1}.$$

Then minimize a differentiable state/position error such as:

\$\$

$\mathcal{L}_{\text{step}}$

$$\begin{aligned}
& |p_{t+1} - p_g|^2 \\
& + \\
& \lambda_v |v_{t+1}|^2.
\end{aligned}$$

\$\$

Determine whether this is supported by literature and whether it is suitable for our system.

Candidate 3 – Multi-step closed-loop trajectory loss

Roll out the UAV for T steps:

$$s_t \rightarrow u_t \rightarrow s_{t+1} \rightarrow u_{t+1} \rightarrow \cdots \rightarrow s_T.$$

Then optimize something like:

\$\$

$\mathcal{L}_{\text{traj}}$

$$\begin{aligned}
& \sum_{t=1}^T \\
& \alpha_t \\
& |p_t - p_g|^2 \\
& + \\
& \lambda_v \\
& \sum_{t=1}^T |v_t|^2 \\
& + \\
& \lambda_u \\
& \sum_{t=0}^{T-1} |u_t|^2.
\end{aligned}$$

\$\$

Determine:

- whether this is differentiable,
- whether backpropagation through the UAV dynamics is possible,
- whether backpropagation through the controller is possible,
- whether this resembles task-oriented control training in the literature,
- computational implications,
- stability issues,
- and whether it is appropriate for our FYP.

Candidate 4 — Terminal navigation loss

Use:

$$\mathbb{E} \left[\mathcal{L}_{\text{terminal}} \right]$$

$$\|p_T - p_g\|^2.$$

Investigate:

- whether this is sufficient,
- whether it can produce poor intermediate trajectories,
- whether it encourages overshoot,
- whether velocity should also be penalized.

Candidate 5 — Hybrid task loss

Investigate whether a combination such as:

$$\mathbb{E} \left[\mathcal{L}_{\text{task}} \right]$$

$$\lambda_1 \mathcal{L}_{\text{terminal}}$$

+

$$\lambda_2 \mathcal{L}_{\text{traj}}$$

+

$$\lambda_3 \mathcal{L}_{\text{control}}$$

where λ_i

would be justified.

Do NOT choose the weights without evidence.

PART 5 — Closed-loop backpropagation

This is critical.

Determine whether our full system can be trained end-to-end:

$$s_t \rightarrow f_\theta \rightarrow z_t \rightarrow \text{channel} \rightarrow \hat{z}_t \rightarrow g_\phi \rightarrow r_t \rightarrow \pi \rightarrow u_t \rightarrow \text{dynamics} \rightarrow s_{t+1}.$$

Analyze differentiability through:

1. encoder,

2. power normalization,
3. AWGN,
4. Rayleigh fading,
5. equalization/CSI,
6. decoder,
7. controller,
8. control clipping,
9. UAV dynamics,
10. multi-step rollout.

For each component, identify whether it is differentiable and whether gradient-based training is practical.

Pay particular attention to:

- hard clipping,
- discrete digital communication baseline,
- success/failure indicators,
- quantization,
- modulation/demodulation,
- stochastic channel noise,
- Rayleigh fading,
- CSI assumptions.

Explain why the digital baseline does NOT need to be trained end-to-end in the same way as DeepJSCC.

PART 6 — Fixed controller vs learned controller

Our project aims to compare:

1. Conventional digital communication
2. Reconstruction-oriented DeepJSCC
3. Task-oriented DeepJSCC

Investigate whether the controller should remain identical across all three methods.

Analyze the fairness implications.

For example:

Digital:

$$s_t \rightarrow \text{digital communication} \rightarrow \hat{s}_t \rightarrow \pi(\hat{s}_t) \rightarrow u_t.$$

Reconstruction DeepJSCC:

$$s_t \rightarrow \text{DeepJSCC} \rightarrow \hat{s}_t \rightarrow \pi(\hat{s}_t) \rightarrow u_t.$$

Task-oriented DeepJSCC:

$$s_t \rightarrow \text{Task-Oriented DeepJSCC} \rightarrow r_t \rightarrow \pi(r_t) \rightarrow u_t.$$

Determine whether using the same fixed controller is necessary/desirable for a fair comparison.

If the task-oriented decoder outputs something other than the full state, explain how the controller can use it fairly.

PART 7 — Training loss vs evaluation metric

Clearly distinguish:

Training objective

A differentiable loss used to optimize:

$$\theta, \phi.$$

from:

Evaluation metrics

Such as:

- success rate,
- final position error,
- time-to-goal,
- path deviation,
- control error,
- failure rate.

Determine whether success rate should be used ONLY for evaluation rather than direct gradient training.

Also determine whether reconstruction MSE should be included in the task-oriented model's training loss.

PART 8 — Communication update frequency

Investigate whether the UAV should transmit:

Every simulation step

$$s_t \rightarrow \textit{communication} \rightarrow \textit{control}$$

or communicate every m steps and hold/interpolate the received representation between transmissions.

Search relevant task-oriented communication/control literature for guidance.

If the literature does not specify this for our UAV model, mark it as project-specific.

PART 9 — Goal-reaching definition

Determine appropriate distinctions between:

Training

Differentiable losses such as:

$$\|p_t - p_g\|^2$$

and

Evaluation

Binary success:

$$\|p_t - p_g\| \leq r_g.$$

Investigate how goal tolerance r_g , maximum episode horizon T , and timeout should be treated.

Do NOT invent numerical values unless a source specifically supports them.

PART 10 — Final mathematical specification

After reviewing the literature, produce a proposed mathematical formulation for our exact FYP.

The final answer MUST contain:

1. State

$$s_t = \dots$$

2. Goal representation

...

3. UAV dynamics

...

4. Controller

...

5. DeepJSCC encoder

$$z_t = f_{\theta}(s_t)$$

6. Channel

AWGN and Rayleigh equations.

7. Decoder

$$r_t = g_{\phi}(\hat{z}_t)$$

8. Control generation

$$u_t = \pi(r_t)$$

9. Closed-loop dynamics

$$s_{t+1} = F(s_t, u_t)$$

10. Task-oriented training loss

Give the exact recommended equation.

11. Reconstruction DeepJSCC loss

Give its equation.

12. Digital baseline pipeline

Give the exact conceptual chain.

13. Evaluation metrics

Give mathematical definitions where appropriate.

14. Training vs evaluation distinction

Clearly explain which quantities are optimized and which are only measured.

PART 11 — Evidence classification table

Create a table with these columns:

Item	Exact literature evidence	Source	Literature-specified?	Project-specific?	Recommended treatment
------	---------------------------	--------	-----------------------	-------------------	-----------------------

Include at least:

- state vector
- goal representation
- UAV dynamics
- controller
- controller gains
- control limits
- decoder output
- task loss
- reconstruction loss
- goal tolerance
- episode horizon
- communication frequency
- AWGN
- Rayleigh fading
- CSI
- power normalization
- bandwidth ratio
- SNR
- training objective
- success metric
- closed-loop training
- number of rollout steps

PART 12 — DO NOT ASSUME LIST

End the report with a section:

“Things we must NOT assume from the literature.”

List every parameter/equation/design choice that is NOT actually specified by the sources and therefore must be selected separately for this FYP.

Be especially careful not to copy:

- CARLA vehicle dynamics,
- autonomous-driving controller parameters,
- driving trajectory definitions,
- driving-specific task losses,
- driving-specific horizons,
- driving-specific normalization,
- driving-specific control limits

into the UAV model without justification.

FINAL REQUIREMENT

The most important output is NOT a generic explanation.

We need a **source-backed, mathematically coherent, implementation-ready formulation** for:

Task-Oriented DeepJSCC + UAV Navigation + Differentiable Closed-Loop Training

If the literature does not provide enough information to make a particular decision, explicitly say:

“NOT SPECIFIED IN SOURCE — PROJECT-SPECIFIC DECISION.”

Do not fill that gap with an invented citation or an unsupported numerical value.

The Diao papers provide strong evidence for **task-oriented communication whose downstream target is a driving action**, and for jointly training a JSCC encoder with a learned driving agent. They do **not** provide a ready-made UAV double-integrator controller, UAV navigation loss, goal radius, rollout horizon, or UAV-specific parameter values. For your FYP, the most defensible formulation is therefore: a fixed, shared goal-reaching PD controller; a task-oriented DeepJSCC decoder that produces a goal-relative navigation representation; and end-to-end training of only the DeepJSCC encoder/decoder through a differentiable closed-loop UAV rollout.

A. Task variable in Diao’s work

$$f_g : \mathcal{Z} \times \mathcal{M} \times \mathcal{H} \rightarrow \mathcal{A} : (\hat{\mathbf{z}}, \mathbf{m}, \mathbf{h}) \mapsto \hat{\mathbf{a}}.$$
$$\min_{f_E, f_a} \mathbb{E}_{\mathbf{a} \sim p(\mathbf{a}|\hat{\mathbf{z}}, \mathbf{m}, \mathbf{h})} [\mathbb{E}_{\hat{\mathbf{a}} \sim p(\hat{\mathbf{a}}|\hat{\mathbf{z}}, \mathbf{m}, \mathbf{h})} [g(\mathbf{a}, \hat{\mathbf{a}})]],$$
$$\min_{\phi, \psi} -I(A; \hat{Z}, M, H) \quad \text{s.t.} \quad I(X; \hat{Z}) - \zeta_{\text{rate}} \leq 0,$$
$$[\mathcal{L}_{\rm IB}]$$

$$-I(A;\hat{Z},M,H) + \beta I(X;\hat{Z}).$$

The first term rewards retention of action-relevant information; the second penalizes information/rate associated with the transmitted representation.^[1]

B. What Diao’s receiver outputs

Diao’s 2025 architecture does not use a conventional source decoder that reconstructs the transmitted camera image. The received/equalized JSCC symbols are fed directly to the driving agent. The agent outputs estimated driving actions and, within DTCP, auxiliary task-critical variables such as features, target velocity, waypoint-related outputs, and predicted control commands.^[1]

This distinction matters for your FYP:

Question	Diao 2025 answer	Implication for the UAV FYP
Does the decoder reconstruct the input image?	No, not as the principal task output	A task-oriented UAV receiver need not reconstruct s_t
Is the task output an action?	Yes; $\hat{a}$ approximates ground-truth a	Direct control prediction has precedent
Are latent/task features used?	Yes, internally in DTCP	A receiver representation r_t is defensible
Are trajectories involved?	Yes, DTCP has a trajectory branch and planned waypoints	Trajectory prediction has precedent, but is driving-specific
Is the plant state rolled out in the stated IB formulation?	Not in the abstract IB objective	A differentiable UAV plant rollout is your extension

C. Diao’s exact task-oriented loss

Literature-supported fact. Diao’s tractable variational information-bottleneck upper bound is

[

$$\mathbb{E}_{\mathbf{a}, \mathbf{x}} \left[\mathbb{E}_{\hat{\mathbf{z}} \mid \mathbf{x}; \boldsymbol{\phi}} \left[\mathbb{E}_{\mathbf{m}, \mathbf{h}} \left[\left[-\log q_{\boldsymbol{\psi}}(\mathbf{a} \mid \hat{\mathbf{z}}, \mathbf{m}, \mathbf{h}) \right] \right] \right] \right]$$

$$+ \beta \mathbb{D}_{\text{KL}} \left(p_{\boldsymbol{\phi}}(\hat{\mathbf{z}} \mid \mathbf{x}) \parallel q(\hat{\mathbf{z}}) \right)$$

]

\right].

]

The empirical minibatch estimate given in the paper is

$$\mathcal{L}_{\text{VIB}} \approx \frac{1}{K_b} \sum_{i=1}^{K_b} \left[-\log q_\psi(a_i \mid \hat{z}_i, m_i, h_i) + \beta D_{\text{KL}}(p_\phi(\hat{z}_i \mid x_i) \parallel q(\hat{z})) \right].$$

Here, $-\log q_\psi(\cdot)$ is the task-prediction term and the KL term is the variational rate/information-bottleneck regularizer.^[1]

Important limitation. Diao’s VIB task term is an action-posterior negative log-likelihood because the driving agent’s action is represented probabilistically. It is not the same as your deterministic UAV goal-reaching loss. Directly copying this term would be unjustified unless you first define a probabilistic model $q_\psi(u_t \mid \hat{z}_t)$ for your UAV controls.

NOT SPECIFIED IN SOURCE — PROJECT-SPECIFIC DECISION: Whether your low-dimensional UAV-state DeepJSCC system should use VIB/KL regularization. Because your project already enforces a fixed channel-use budget and power constraint, a trajectory/control objective without a VIB term is mathematically valid. A VIB extension can be reported as an ablation, not as a required component of task-oriented DeepJSCC.

D. DTCP: what it is—and is not

Literature-supported fact. DTCP means **Delay-aware Trajectory-guided Control Prediction**. It was designed for Diao’s delayed, edge-enabled autonomous-driving system, where camera data become stale during uplink transmission, edge inference, downlink communication, and control execution.^[1]

DTCP combines two learned branches:

- A **trajectory branch** extracts trajectory features, autoregressively predicts future waypoint-related information, and uses a low-level PID controller to generate trajectory commands.
- A **control branch** predicts a sequence of future control commands using recurrent processing and information from the trajectory branch.
- A conditional combination rule fuses trajectory-branch and control-branch commands based on driving context, delay, and a threshold.^[1]

The paper defines end-to-end delay as

[

\delta

\delta_e + \delta_u + \delta_a + \delta_d + \delta_c,

]

where encoding, uplink, agent computation, downlink, and control execution delays are included. A prediction horizon l_p is selected to compensate perceived delay, which is described

as $\delta - l_p$.^[1]

Mathematical interpretation. DTCP is more than one-step action prediction. It predicts sequences of future driving commands, and its trajectory branch predicts waypoint-related outputs. It is learned and co-designed with communication, but its architecture, timing model, CARLA vehicle interface, route representation, steering/throttle/brake command format, turning condition, and PID details belong to the autonomous-driving problem.

Do not transfer this directly. For your simulator there is no stated communication delay model, no CARLA image stream, no waypoint route planner, and no vehicle steering/braking action space. Therefore:

NOT SPECIFIED IN SOURCE — PROJECT-SPECIFIC DECISION: whether to model communication/computation delay and use delay-compensating action or trajectory prediction for the UAV.

For the initial FYP scope, do not implement DTCP. It would add an additional research problem—delay-aware predictive control—rather than isolating task-oriented source-channel coding.

Representation and controller choice

Options for the UAV receiver

Your physical state is

$$s_t = [x_t, y_t, v_{x,t}, v_{y,t}, x_g, y_g]^\top,$$

with

$$p_t = [x_t, y_t]^\top, \quad v_t = [v_{x,t}, v_{y,t}]^\top, \quad p_g = [x_g, y_g]^\top.$$

The following table distinguishes literature precedent from suitability for your exact FYP.

Option	Receiver output	Relevant precedent	Natural training target	Differentiable?	FYP assessment
A: full reconstruction	$\hat{s}_t = [\hat{p}_t, \hat{v}_t, \hat{p}_g]^\top$	Reconstruction-oriented DeepJSCC reconstructs its source using MSE	State reconstruction MSE, optionally followed by fixed control	Yes	Necessary as the reconstruction DeepJSCC baseline; not the strongest task-oriented design

Option	Receiver output	Relevant precedent	Natural training target	Differentiable?	FYP assessment
B: goal-relative representation	$r_t = [\hat{e}_{p,t}, \hat{v}_t]^\top$, ideally representing $[p_g - p_t, v_t]$	Functional/task-sufficient representation follows the task-oriented communication principle; Diao uses task features rather than full image reconstruction	Closed-loop navigation cost, representation loss, or action imitation	Yes	Recommended primary task-oriented design
C: direct control	$\hat{u}_t$	Diao's task variable is an action and its agent predicts driving commands	$\ \hat{u}_t - u_t^*\ ^2$ or a rollout navigation cost	Yes, if output activation is differentiable	Defensible but mixes communication quality with a learned controller/policy
D: trajectory or command sequence	$\hat{p}_{t+1:t+H}$ or $\hat{u}_{t:t+H-1}$	Diao DTCP predicts future commands and has a trajectory branch	Multi-horizon trajectory/control loss	Yes	Possible extension, but excessive for the initial no-delay double-integrator scope

Recommended task representation

For your core experiment, define a deterministic task map

[
 $q(s_t)$
 r_t^*

$$\begin{bmatrix} e_{p,t} \\ v_t \end{bmatrix}$$

$$\begin{bmatrix} p_g - p_t \\ v_t \end{bmatrix}$$

$$\begin{bmatrix} x_g - x_t \\ y_g - y_t \\ v_{x,t} \\ v_{y,t} \end{bmatrix}$$

.
]

The task-oriented decoder produces

$$\begin{bmatrix} \hat{r}_t \\ g_{\phi}(\hat{z}_t) \end{bmatrix}$$

$$\begin{bmatrix} \hat{e}_{p,t} \\ \hat{v}_t \end{bmatrix}$$

The receiver then applies the same fixed controller mapping to $\hat{r}_t$ that an ideal state-feedback controller would apply to r_t^* .

This is **not** a claim that Diao specifies this UAV representation. It is the cleanest project adaptation because:

- It removes the absolute-coordinate redundancy inherent in transmitting p_t and p_g separately when the navigation law depends on their difference.
- It retains the velocity information necessary for damping.
- It permits a fixed and transparent controller.
- It gives a direct, fair comparison against reconstructed-state methods.
- It keeps the research focus on whether DeepJSCC preserves what navigation needs under AWGN and fading.

NOT SPECIFIED IN SOURCE – PROJECT-SPECIFIC DECISION: the exact UAV task representation $r_t^* = [p_g - p_t, v_t]^\top$.

Fixed PD controller

For the continuous-time double integrator

$$\dot{p}(t) = v(t), \quad \dot{v}(t) = u(t),$$

the goal equilibrium is $p = p_g$, $v = 0$. Define

$$e_p = p_g - p, \quad e_v = -v.$$

The candidate controller

$$u^* = K_p e_p + K_d e_v$$

is equivalent to the familiar regulation law

$$u^* = -K_p(p - p_g) - K_d v.$$

For scalar gains applied identically on both axes, use

$$K_p = k_p I_2, \quad K_d = k_d I_2.$$

Without saturation, the error dynamics are

$$\ddot{e}_p + K_d \dot{e}_p + K_p e_p = 0.$$

Thus, positive definite K_p and K_d yield the usual damped second-order regulation structure. PD feedback is a standard stabilizing feedback structure for a double integrator.^{[2] [3]}

For the discrete-time simulation, use the stated dynamics:

$$\begin{aligned} p_{t+1} &= p_t + \Delta t \, v_t, \\ v_{t+1} &= v_t + \Delta t \, u_t. \end{aligned}$$

The operational control law is

$$\begin{bmatrix} \hat{u}_t \\ \hat{v}_t \end{bmatrix} = -K_p \hat{e}_{p,t} - K_d \hat{v}_t,$$

$$\hat{u}_t = \begin{cases} \hat{u}_t, & \text{if } |\hat{u}_t| \leq u_{\max} \\ u_{\max} \operatorname{sign}(\hat{u}_t), & \text{if } |\hat{u}_t| > u_{\max} \end{cases}$$

$$\hat{v}_t = \begin{cases} \hat{v}_t, & \text{if } |\hat{v}_t| \leq v_{\max} \\ v_{\max} \operatorname{sign}(\hat{v}_t), & \text{if } |\hat{v}_t| > v_{\max} \end{cases}$$

$$\hat{u}_t = \begin{cases} \hat{u}_t, & \text{if } |\hat{u}_t| \leq u_{\max} \\ u_{\max} \operatorname{sign}(\hat{u}_t), & \text{if } |\hat{u}_t| > u_{\max} \end{cases}$$

The sign is correct because the received velocity estimate $\hat{v}_t$ is the actual velocity representation; damping requires $-K_d \hat{v}_t$. If instead the decoder output is defined as $\hat{e}_{v,t} \approx -v_t$, then the equivalent form is $K_p \hat{e}_{p,t} + K_d \hat{e}_{v,t}$.

Controller decisions

Question	Defensible conclusion
Is PD appropriate?	Yes, for the stated planar double-integrator goal-regulation model
Is it sufficient?	Yes, for a simulation-only FYP that isolates communication effects; it is not a full quadrotor flight controller
Does Diao specify gains for this UAV?	No
Should K_p, K_d be fixed?	Yes, for the primary benchmark

Question	Defensible conclusion
Should the controller be learned jointly?	Not for the primary comparison
Why fixed?	It prevents task-oriented DeepJSCC from gaining performance merely by using a more powerful controller than its baselines
Can learned control be added?	Yes, as a clearly labeled separate ablation, not as the fair main comparison

PROJECT-SPECIFIC PARAMETERS — TO BE SELECTED AND JUSTIFIED SEPARATELY: K_p , K_d , $u_{\max}$, any $v_{\max}$, and Δt .

A principled controller-selection procedure is to choose these values first using a noiseless, full-state, perfect-communication simulator; verify stable convergence for the training/test initial-state and goal distribution; freeze them; then use the same values for all communication methods. That is a methodology recommendation, not a parameter prescription from Diao.

Recommended closed-loop formulation

1. State, goal, and plant

Use

$$\begin{bmatrix} p_t \\ v_t \\ p_g \end{bmatrix}$$

$$\begin{bmatrix} x_t \\ y_t \\ v_{x,t} \\ v_{y,t} \\ x_g \\ y_g \end{bmatrix}$$

$\in \mathbb{R}^6$.

Use the task variable

$$[r_t^{\star} = q(s_t)$$

$$\begin{bmatrix} p_g - p_t \\ v_t \end{bmatrix}$$

$$\in \mathbb{R}^4.$$

The discrete UAV plant is

$$[s_{t+1} = F(s_t,u_t)$$

$$\begin{bmatrix} p_t + \Delta t \, v_t \\ v_t + \Delta t \, u_t \\ p_g \end{bmatrix}$$

$$\cdot$$

The goal is fixed within an episode:

$$p_{g,t+1} = p_{g,t} = p_g.$$

2. Task-oriented DeepJSCC

A clean task-oriented transmitter can take the full physical state:

$$\tilde{z}_t = f_{\theta}(s_t) \in \mathbb{R}^{2k}.$$

Pair real components into k complex symbols and normalize to a per-codeword average-power target P :

$$z_t = \sqrt{kP} \frac{\tilde{z}_t}{\|\tilde{z}_t\|_2}, \quad z_t \in \mathbb{C}^k.$$

This is the same normalization principle used in the original DeepJSCC formulation, which maps a source to complex channel symbols subject to an average power constraint.^[4]

For AWGN,

$$\tilde{z}_t^{\text{ch}} = z_t + n_t, \quad n_t \sim \mathcal{CN}(0, \sigma_n^2 I_k).$$

For flat Rayleigh fading,

$$\tilde{z}_t^{\text{ch}} = h_t \odot z_t + n_t, \quad h_t \sim \mathcal{CN}(0, I_k).$$

Under a perfect-CSI equalization assumption,

$$\left[\hat{\mathbf{z}}_t\right.$$

$$\frac{h_t^{\ast}|h_t|^2}{\odot} \tilde{z}_t^{\rm ch}.$$

$$\left.] \right.$$

Diao’s 2025 model likewise uses a complex fading channel,

$$\check{z} = h \cdot z + n,$$

and assumes perfect CSI for zero-forcing equalization,

$$\hat{z} = \frac{h^*}{|h|^2} \check{z}.$$

However, Diao uses a frequency-selective OFDM model for the driving system, not your scalar/flat Rayleigh UAV channel.^[1]

The task-oriented decoder is

$$\left[\hat{\mathbf{r}}_t\!=\!\mathbf{g}_\phi(\hat{\mathbf{z}}_t)\right.$$

$$\begin{bmatrix} \hat{e}_{p,t} \\ \hat{v}_t \end{bmatrix}$$

$$\cdot$$

$$\left.] \right.$$

The fixed controller is

$$\left[\tilde{u}_t\right.$$

$$K_p\hat{e}_{p,t}\!-\!K_d\hat{v}_t,$$

$$\left.] \right.$$

$$u_t = \text{clip}(\tilde{u}_t, -u_{\max}, u_{\max}).$$

Thus the complete trainable loop is

$$s_t \rightarrow f_\theta \rightarrow z_t \rightarrow \text{channel} \rightarrow \hat{z}_t \rightarrow g_\phi \rightarrow \hat{r}_t \rightarrow \pi \rightarrow u_t \rightarrow F \rightarrow s_{t+1}.$$

3. Recommended task-oriented loss

The recommended primary loss is a finite-horizon closed-loop navigation objective:

$$\begin{aligned} & \mathbb{E}_{\xi} \left[\right. \\ & \quad \boxed{ \\ & \quad \mathcal{L}_{\text{TO}} \\ & \quad \mathbb{E}_{\xi} \left[\right. \\ & \quad \quad \sum_{t=1}^T \alpha_t \\ & \quad \quad |p_t - p_g|^2 \\ & \quad \quad + \\ & \quad \quad \lambda_v \\ & \quad \quad \sum_{t=1}^T |v_t|^2 \\ & \quad \quad + \\ & \quad \quad \lambda_u \\ & \quad \quad \sum_{t=0}^{T-1} |u_t|^2 \\ & \quad \quad + \\ & \quad \quad \lambda_T \\ & \quad \quad |p_T - p_g|^2 \\ & \quad \quad + \\ & \quad \quad \lambda_{v,T} \\ & \quad \quad |v_T|^2 \\ & \quad \left. \right] \\ & \left. \right] \end{aligned}$$

subject to the closed-loop system above.

Here, ξ denotes random episode variables: initial state, goal, AWGN samples, fading coefficients, and possibly sampled SNR. The terminal position term is intentionally retained even though the running position term already includes $t = T$; it lets you explicitly emphasize terminal goal reaching if desired.

This is an implementation-ready equation, but its **specific weights are not supplied by Diao or by a source for your exact UAV model**:

NOT SPECIFIED IN SOURCE — PROJECT-SPECIFIC DECISION: α_t , λ_v , λ_u , λ_T , and $\lambda_{v,T}$.

This choice is mathematically defensible because differentiable predictive-control literature explicitly uses automatic differentiation through closed-loop dynamics and constraint penalties to learn policies/parameters. It is also aligned in spirit with task-oriented

communication: optimize downstream control performance rather than source reconstruction. It is not an equation copied from Diao. [5]

4. Candidate losses: decision

Candidate	Status	Assessment
1. Control imitation	$\frac{1}{T} \sum_t \ \hat{u}_t - u_t^*\ ^2$	Literature-aligned through Diao’s action-centered formulation, but this exact UAV loss is a project-specific adaptation
2. One-step navigation loss	$\ p_{t+1} - p_g\ ^2 + \lambda_v \ v_{t+1}\ ^2$	Differentiable but myopic; not recommended as the only objective
3. Multi-step rollout loss	Running position, velocity, effort, and terminal terms	Recommended primary loss; differentiable through your analytic dynamics
4. Terminal-only loss	$\ p_T - p_g\ ^2$	Differentiable but insufficient alone; it can permit poor intermediate behavior and nonzero terminal velocity
5. Hybrid loss	Terminal + rollout + imitation	Valid only if each term has a clear role; weights require tuning and ablation

Candidate 1: control imitation

Let the noiseless full-state expert command be

[
 u_t^*

$$\left(\frac{1}{T} \sum_{t=0}^{T-1} \|K_p(p_g - p_t) - K_{dv} \dot{v}_t - u_{\max}\|_{u_{\max}}^2 \right)$$

If the task decoder produces a control estimate directly, then

[
$$\frac{1}{T} \sum_{t=0}^{T-1} \|\hat{u}_t - u_t^*\|^2$$

For the recommended representation-based decoder, define instead

[
 $\hat{u}_t^{\text{pre}}$

$K_p \hat{e}_{p,t} - K_d \hat{v}_t,$
]

[
 $\mathcal{L}_{\text{control}}$

$\frac{1}{T}$
 $\sum_{t=0}^{T-1}$
 $|\hat{u}_t^{\text{pre}} - u_t^{\text{star}}|_2^2.$
]

This is a useful optional auxiliary loss because Diao’s task target is action prediction. But the exact UAV equation is not specified by Diao.

NOT SPECIFIED IN SOURCE — PROJECT-SPECIFIC DECISION: whether to include $\mathcal{L}_{\text{control}}$, and its coefficient.

Candidate 2: one-step loss

Because

$$p_{t+1} = p_t + \Delta t v_t$$

does not contain u_t , a position-only one-step term

$$\|p_{t+1} - p_g\|_2^2$$

does not directly provide a gradient to u_t under your specified forward-Euler position update. The control affects v_{t+1} , which affects p_{t+2} . Therefore it is a particularly weak standalone objective.

A one-step loss becomes more control-sensitive if you evaluate the next velocity,

[
 $\mathcal{L}_{\text{step}}$

$|p_{t+1} - p_g|^2$
 $+$
 $|\lambda v|v_{t+1}|_2^2,$
]

but it remains short-sighted. Use it only as an auxiliary diagnostic or an initial curriculum stage if later justified experimentally.

Candidate 3: multi-step rollout loss

This is the strongest choice for your model. The encoder/decoder must preserve the state information that enables repeated navigation decisions under noisy channels, not merely one action at one time instant.

Backpropagation is available because:

$$\left[\frac{\partial v_{t+1}}{\partial u_t} \right]$$

$\Delta t L_2,$

]

$$\left[\frac{\partial p_{t+1}}{\partial v_t} \right]$$

$\Delta t L_2.$

]

The effect of u_t reaches position at the subsequent step in the stated discretization. Autograd can therefore propagate gradients through unrolled state transitions, controller operations, and neural networks.

The practical costs are:

- Memory grows approximately with rollout length T , because intermediate activations must be retained.
- Long rollouts can create exploding or vanishing gradients.
- Saturation can create regions with zero or weak gradient.
- Random fading/noise makes gradients stochastic; train using Monte Carlo channel samples and minibatch averaging.
- A curriculum from short to longer horizons may help, but this is **NOT SPECIFIED IN SOURCE — PROJECT-SPECIFIC DECISION**.

Candidate 4: terminal loss

$$\left[\mathcal{L}_{\text{terminal}} \right]$$

$|p_T - p_g|^2$

]

is differentiable, but it alone does not penalize:

- high speed at arrival,

- large detours,
- oscillations,
- excessive actuation,
- unsafe or unstable intermediate behavior.

Add terminal velocity:

$$[\mathcal{L}_{\text{terminal}} + v]$$

$$|p_T - p_g|^2 +$$

$$\lambda_{v,T} |v_T|^2.$$

Even then, a running trajectory term is preferable for controlled, non-oscillatory convergence.

Candidate 5: hybrid loss

A transparent optional extension is

$$[\mathcal{L}_{\text{hybrid}}$$

$$\mathcal{L}_{\text{TO}}$$

$$+$$

$$\lambda_{\text{ctrl}} \mathcal{L}_{\text{control}}.$$

$$]$$

Use this only after establishing the primary rollout-only model. It tests whether expert-command supervision improves robustness and convergence. Do not set or report λ_{ctrl} as literature-derived.

5. Reconstruction DeepJSCC baseline

The reconstruction baseline must reconstruct the full state, using the same channel-use budget, channel distribution, SNR convention, power target, and controller.

$$z_t^{\text{rec}} = f_{\theta_{\text{rec}}}(s_t),$$

$$\hat{s}_t = g_{\phi_{\text{rec}}}(\hat{z}_t^{\text{rec}}),$$

[

$$\mathcal{L}_{\text{rec}}$$

$$\mathbb{E}_{\xi} \left[\frac{1}{T} \sum_{t=0}^{T-1} |D_s^{-1}(\bar{s}_t - \hat{s}_t)|^2 W_s \right].$$

Here $\bar{s}_t$ denotes the normalized state used by the network, D_s reverses normalization if you want physical-unit weighting, and $W_s \succeq 0$ is a stated state-component weighting matrix. For a simplest baseline:

[

$$\mathcal{L}_{\text{rec}}$$

$$\frac{1}{T} \sum_{t=0}^{T-1} |\bar{s}_t - \hat{s}_t|^2.$$

The original DeepJSCC formulation uses average MSE between source input and reconstruction; transferring that principle to a state vector is a valid reconstruction-baseline adaptation, not an equation literally printed for UAV states. ^[4]

The controller receives

[

$$\hat{\mathbf{e}}_{p,t}$$

$$\hat{\mathbf{p}}_{g,t} - \hat{\mathbf{p}}_t,$$

$$\hat{\mathbf{v}}_t,$$

and applies

$$u_t = \text{clip} \left(K_p (\hat{p}_{g,t} - \hat{p}_t) - K_d \hat{v}_t, -u_{\max}, u_{\max} \right).$$

NOT SPECIFIED IN SOURCE — PROJECT-SPECIFIC DECISION: state normalization, whether state MSE is physically weighted, and the selected state-loss scaling.

6. Digital baseline

The digital baseline is a fixed separated chain:

$s_t \rightarrow$ state normalization $\rightarrow$ quantization $\rightarrow$ source packet $\rightarrow$ channel coding $\rightarrow$ modulation $\rightarrow$

It does not need an end-to-end gradient because its quantization, coding, modulation, demodulation, and decoding blocks are predefined algorithms, not neural parameters being optimized. It is evaluated over the same channel/SNR conditions and an explicitly matched communication budget.

NOT SPECIFIED IN SOURCE – PROJECT-SPECIFIC DECISION: quantizer design, bit allocation, packetization, error-correcting code, modulation order, code rate, outage/packet-loss behavior, retransmission policy, and exact budget-matching rule for a six-dimensional state.

Differentiability and fair evaluation

Gradient path audit

Component	Differentiable?	Practical training judgment
Neural encoder f_θ	Yes	Standard backpropagation
Complex-to-real conversion	Yes	A reshape/pairing operation; no mathematical obstacle
Power normalization	Yes except at zero norm	Add a numerically safe ϵ in implementation; ϵ is a numerical implementation choice
AWGN injection	Yes with respect to input for a sampled noise realization	Use reparameterized sampling $n = \sigma\epsilon$, $\epsilon \sim \mathcal{CN}(0, I)$
Rayleigh fading multiplication	Yes with respect to z_t for sampled h_t	Sample h_t during training; gradients flow through multiplication
Perfect-CSI equalization	Yes almost everywhere when $h_t \neq 0$	Deep fades can amplify noise and gradients; use a clearly declared regularized equalizer if needed
Decoder g_ϕ	Yes	Standard backpropagation
Fixed PD controller	Yes	Linear in decoder outputs before saturation
Hard clipping	Piecewise differentiable; derivative is zero in saturated regions	Usable, but can impair learning
UAV plant F	Yes	Affine dynamics; ideal for closed-loop backpropagation
Multi-step rollout	Yes	Memory/gradient stability become practical constraints
Binary success indicator	No	Evaluation only
Quantization	Not ordinarily differentiable	Do not use it in the end-to-end DeepJSCC training path

Component	Differentiable?	Practical training judgment
Digital coding/modulation chain	Generally nondifferentiable/discrete	Evaluate algorithmically; do not force joint neural training

Saturation

For evaluation, preserve the physical actuator constraint:

$$u_t = \text{clip}(\tilde{u}_t, -u_{\max}, u_{\max}).$$

Hard clipping has a derivative of zero outside the unsaturated interval. This does not make training impossible, but it can stall learning when the decoder repeatedly generates commands beyond limits.

You have two defensible choices:

- Train and evaluate with hard clipping, while monitoring saturation frequency.
- Train with a smooth bounded actuator map such as

$$u_t = u_{\max} \tanh(\tilde{u}_t / u_{\max}),$$

then evaluate with the same smooth map.

The second choice changes the controller nonlinearity. Therefore:

NOT SPECIFIED IN SOURCE — PROJECT-SPECIFIC DECISION: hard clipping versus a smooth bounded control parameterization.

Do not train with tanh and silently evaluate with a different hard-clipped plant; if you do compare them, report it as an ablation.

CSI and fading

Your Rayleigh experiments need an explicit receiver assumption:

- **Perfect CSI:** receiver knows h_t and equalizes. This is a clean initial benchmark.
- **No CSI:** decoder receives raw $\tilde{z}_t^{\text{ch}}$ and must implicitly handle fading.
- **Estimated CSI:** receiver uses $\hat{h}_t$, introducing channel-estimation error.

Diao explicitly assumes perfect CSI in its channel equalization equation. That supports perfect CSI as a benchmark assumption, but not as a claim of UAV realism. ^[1]

NOT SPECIFIED IN SOURCE — PROJECT-SPECIFIC DECISION: CSI acquisition, pilot overhead, estimation error, and whether CSI is appended to the decoder input.

Fair controller rule

For the main comparison, use one fixed controller design and one set of frozen gains:

Method	Receiver output	Controller interface
Digital	$\hat{s}_t$	Compute $\hat{e}_{p,t} = \hat{p}_{g,t} - \hat{p}_t$, use $\pi(\hat{e}_{p,t}, \hat{v}_t)$
Reconstruction DeepJSCC	$\hat{s}_t$	Exactly the same mapping $\pi(\hat{p}_{g,t} - \hat{p}_t, \hat{v}_t)$
Task-oriented DeepJSCC	$\hat{r}_t = [\hat{e}_{p,t}, \hat{v}_t]$	The same PD mapping $\pi(\hat{e}_{p,t}, \hat{v}_t)$

This is fair because all methods ultimately use the same control law. The task-oriented model is allowed to avoid reconstructing task-irrelevant information, but it is not allowed to use a more capable learned policy.

A directly learned-control decoder, $\hat{u}_t = g_\phi(\hat{z}_t)$, should be treated as a separate experiment because it changes both the communication representation and the control policy.

Training versus evaluation

The binary success event is

$$\left[\begin{array}{l} \mathbf{1} \\ \exists t \in \{0, \dots, T\}: \\ |p_t - p_g|_2 \leq r_g \end{array} \right]$$

Across N test episodes,

$$\left[\begin{array}{l} \frac{1}{N} \sum_{i=1}^N \\ \text{operatorname{Succ}}^{\{i\}} \end{array} \right]$$

This indicator is non-differentiable and must be used as an **evaluation metric**, not the primary gradient-training loss.

Recommended evaluation quantities are:

[
e_{\rm final}

|p_T-p_g|_2,
]

[
e_{\rm min}

\min_{\{0\leq t\leq T\}}|p_t-p_g|_2,
]

[
t_{\rm goal}

\min
\left\{
t:
|p_t-p_g|_2\leq r_g
\right\},
]

with timeout recorded if the set is empty,

[
J_u

\sum_{t=0}^{T-1}
|u_t|_2^2,
]

[
J_v

\sum_{t=1}^T
|v_t|_2^2,
]

and, if desired, communication-relevant measures such as state/representation MSE, average channel uses, and saturation frequency.

PROJECT-SPECIFIC PARAMETERS – TO BE SELECTED AND JUSTIFIED SEPARATELY:
goal radius r_g , horizon T , timeout definition, initial-state distribution, goal distribution,
and whether success requires stopping velocity below a specified tolerance.

Communication update frequency

The clean initial protocol is one transmission and control update at every simulation step:

$$s_t \rightarrow \text{communication} \rightarrow u_t \rightarrow s_{t+1}.$$

This makes the effect of communication noise observable without adding age-of-information, zero-order hold, intermittent observation, or latency-compensation confounders.

Diao's DTCP uses time slots and explicitly models delayed observations/commands for an edge-enabled driving scenario. It does not specify whether your UAV should communicate every m double-integrator steps.^[1]

NOT SPECIFIED IN SOURCE — PROJECT-SPECIFIC DECISION: communication period m , zero-order hold versus interpolation, age-of-information model, and whether delay is in scope.

If you later introduce communication every $m > 1$ steps, define it explicitly:

$$u_{t+j} = \pi(\hat{r}_t), \quad j = 0, \dots, m-1,$$

unless a local predictor or controller is added. That is a separate intermittent-communication experiment, not part of the initial comparison.

Evidence classification and non-assumptions

Evidence classification table

Item	Exact literature evidence	Source	Literature-specified?	Project-specific?	Recommended treatment
State vector	Diao uses image input plus driving state information; not a six-state UAV vector	Diao 2025	No	Yes	Use the stated $s_t = [x_t, y_t, v_{x,t}, v_{y,t}, x_g, y_g]^\top$
Goal representation	Diao state information includes destination coordinates, but no goal-relative UAV vector	Diao 2025	No	Yes	Use $r_t^* = [p_g - p_t, v_t]^\top$
UAV dynamics	Diao uses CARLA autonomous driving, not a planar double integrator	Diao 2025	No	Yes	Use the supplied discrete double-integrator plant

Item	Exact literature evidence	Source	Literature-specified?	Project-specific?	Recommended treatment
Controller	Diao DTCP includes learned branches and a low-level PID in the trajectory branch	Diao 2025	Not for UAV	Yes	Use a fixed PD controller
Controller gains	No gains for your plant	Diao 2025	No	Yes	Freeze after noiseless calibration
Control limits	No UAV $u_{\max}$	Diao 2025	No	Yes	Define and justify separately
Decoder output	Diao agent outputs estimated driving action and DTCP task variables	Diao 2025	Yes, for driving	Yes, for UAV form	Decode goal-relative state representation
Task loss	Action distortion objective; VIB negative log posterior plus KL term	Diao 2025	Yes, for driving formulation	Yes, for UAV rollout loss	Use rollout navigation cost; optionally test action imitation
Reconstruction loss	Original DeepJSCC uses source reconstruction MSE	Bourtsoulatze et al.	Yes, for images	Yes, for state vector	Use full-state MSE baseline
Goal tolerance	No UAV goal radius	Diao / DeepJSCC	No	Yes	Evaluation-only r_g
Episode horizon	DTCP prediction horizon is delay-related, not a UAV goal-reaching horizon	Diao 2025	No	Yes	Define T separately
Communication frequency	Diao uses time slots and a delay model; no UAV m	Diao 2025	No	Yes	Start with every step
AWGN	Complex Gaussian channel noise is used	Diao 2025; DeepJSCC	Yes	No, for basic model	Use stated complex AWGN convention consistently

Item	Exact literature evidence	Source	Literature-specified?	Project-specific?	Recommended treatment
Rayleigh fading	Diao uses a frequency-selective OFDM fading model, not necessarily i.i.d. flat Rayleigh	Diao 2025	Partially	Yes	State flat Rayleigh model separately
CSI	Perfect CSI and zero-forcing equalization assumed	Diao 2025	Yes	Yes, for realism level	Use perfect CSI benchmark, then ablate if feasible
Power normalization	Original DeepJSCC normalizes symbols to satisfy average power constraint	Bourtsoulatze et al.	Yes	No, principle	Apply per-codeword normalization
Bandwidth ratio	Diao defines l_z/l_x ; DeepJSCC defines channel uses relative to source dimension	Diao 2025; Bourtsoulatze et al.	Yes	Yes, for low-dimensional state budget	Define exact $k/6$ convention
SNR	Channel-noise and power conventions are model-dependent	DeepJSCC / Diao	Partially	Yes	Freeze one convention for all systems
Training objective	Diao jointly optimizes communication and driving agent; no UAV plant rollout objective stated	Diao 2025	Yes, broadly	Yes, exact equation	Use $\mathcal{L}_{\text{TO}}$ above
Success metric	Diao reports driving score; no UAV binary radius success	Diao 2025	No	Yes	Use success rate only at evaluation
Closed-loop training	Diao jointly trains JSCC and agent; not an explicit UAV-dynamics BPTT formulation	Diao 2025	Partially	Yes	Backpropagate through analytic F

Item	Exact literature evidence	Source	Literature-specified?	Project-specific?	Recommended treatment
Rollout steps	Diao's DTCP horizons compensate delay; not an FYP navigation rollout length	Diao 2025	No	Yes	Select T and report sensitivity
Digital baseline	DeepJSCC literature compares against separated coding; no six-state UAV packet standard is specified	Bourtsoulatze et al.	Partially	Yes	Define a matched-budget fixed digital chain
Learned controller	Diao jointly trains a learned driving agent	Diao 2025	Yes, for driving	Yes, for fair UAV benchmark	Keep controller fixed in main study

Diao's thesis explicitly frames task-oriented communication as transmitting only information needed for downstream machine decision-making rather than full data fidelity, and identifies the autonomous-driving work as an application-specific communication/control co-design. The thesis and 2025 paper support your conceptual direction, but neither fills the UAV-specific gaps listed above. ^[6]

Things we must NOT assume from the literature

- Do **not** assume Diao's CARLA vehicle dynamics are valid for a UAV.
- Do **not** reuse CARLA steering, throttle, brake, waypoint, route, lane-following, collision, or driving-score definitions.
- Do **not** transfer Diao's trajectory branch, control GRU, image-attention blocks, or DTCP architecture into the low-dimensional UAV-state problem without separately justifying them.
- Do **not** treat Diao's delay threshold, delay decomposition, conditional branch-combination rule, or prediction horizon as UAV parameters.
- Do **not** claim Diao's action-posterior VIB loss is your UAV navigation loss.
- Do **not** claim that Diao specifies a double-integrator UAV controller.
- Do **not** invent K_p , K_d , $u_{\max}$, $v_{\max}$, Δt , r_g , or T , or label any selected values "from Diao."
- Do **not** assume that decoding the full source state is required for task-oriented communication; Diao's architecture instead feeds received JSCC symbols to a task agent.
- Do **not** assume that direct control prediction is automatically fair against state-reconstruction baselines; it changes the control architecture unless carefully controlled.

- Do **not** assume that a terminal-only objective ensures safe, smooth, or low-energy navigation.
- Do **not** use binary success indicators as gradient objectives.
- Do **not** assume every-step transmission, communication period m , packet hold behavior, latency, or age-of-information behavior from Diao's driving setup.
- Do **not** assume perfect CSI is realistic merely because it is used in Diao; state it as a benchmark assumption.
- Do **not** copy Diao's OFDM frequency-selective fading model into an i.i.d. flat-Rayleigh experiment without explicitly redefining the channel.
- Do **not** compare methods under different channel-use budgets, unequal SNR definitions, unequal power constraints, different controller gains, or different initial/goal distributions.
- Do **not** train the digital baseline end-to-end merely to make its training pipeline look similar to DeepJSCC; its role is a fixed algorithmic separated-communication comparator.
- Do **not** add a reconstruction MSE term to the primary task-oriented model by default. If included, it must be a separately motivated multi-objective ablation, because it partly changes the method back toward reconstruction-oriented communication.
- Do **not** present the recommended goal-relative representation or rollout loss as though they were copied from Diao. They are the proposed, mathematically coherent adaptation for this FYP.

**

1. <https://www.arxiv.org/pdf/2503.08661.pdf>
2. <https://lewisgroup.uta.edu/ee4343/lectures99/pidLL.pdf>
3. <https://scispace.com/pdf/time-sub-optimal-nonlinear-pi-and-pid-controllers-applied-to-1irc0ifzjo.pdf>
4. https://discovery.ucl.ac.uk/id/eprint/10077850/1/JSCC_AE_ICASSP2019.pdf
5. <https://arxiv.org/abs/2004.11184>
6. <https://theses.gla.ac.uk/85353/1/2025DiaoPhD.pdf>
7. <https://www.gla.ac.uk/schools/computing/staff/emmal/>
8. <https://www.diva-portal.org/>
9. <https://www.gla.ac.uk/schools/computing/research/researchsections/systems-section/news/>
10. <https://orcid.org/0000-0003-4200-0669>
11. <https://theses.gla.ac.uk/view/subjects/T1.html>
12. <https://theses.gla.ac.uk/85353/>
13. <https://yufeng-diao.github.io/publications/>
14. <https://yufeng-diao.github.io/>
15. <https://www.linkedin.com/in/yufeng-diao-b96552255>
16. <https://socialcdt.org/students/aligned-students/>
17. <https://ieeexplore.ieee.org/document/10620735>

18. https://www.imperial.ac.uk/media/imperial-college/research-centres-and-groups/ipc-lab/JGM_ICASSP20.pdf
19. <https://www.gla.ac.uk/pgrs/xiangminxu/>
20. <https://dblp.dagstuhl.de/rec/journals/tccn/BourtsoulatzeKG19.html>
21. <https://web3.arxiv.org/pdf/2508.00715>
22. <https://www.research-collection.ethz.ch/bitstream/handle/20.500.11850/402967/64.pdf?sequence=1&isAllowed=y>
23. <https://ar5iv.labs.arxiv.org/html/2502.15472>
24. <https://cs-conference-ranking.org/author/yufeng-diao>
25. <https://cs-conference-ranking.org/author/zhen-meng>
26. <https://cs-conference-ranking.org/author/changyang-she>
27. https://rpg.ifi.uzh.ch/docs/CoRL24_Yunlong.pdf
28. <https://ifatwww.et.uni-magdeburg.de/ifac2020/media/pdfs/3112.pdf>
29. https://courses.grainger.illinois.edu/ece486/fa2025/documentation/lectures/slides_2014/lec12.pdf
30. <https://arxiv.org/html/2512.19177v1>
31. <https://www.research-collection.ethz.ch/server/api/core/bitstreams/b5cf1b9d-abc9-499e-b648-e7bf6d0f3557/content>
32. https://lo.unisa.edu.au/pluginfile.php/3833762/mod_glossary/attachment/42392/l_pid.pdf
33. <https://www.roboticsproceedings.org/rss20/p003.pdf>
34. <https://openreview.net/pdf?id=ryxC6kSYPr>
35. <https://mediatum.ub.tum.de/doc/1539142/document.pdf>
36. https://skoge.folk.ntnu.no/prost/proceedings/dycops-2025/DYCOPS2025_offline_program/contents/papers/0007.pdf
37. [https://feng.stafpu.bu.edu.eg/Electrical Engineering/871/crs-18473/Files/EEC_321_Automatic Control \(1\)_Lect\(2\)_2022.pdf](https://feng.stafpu.bu.edu.eg/Electrical Engineering/871/crs-18473/Files/EEC_321_Automatic Control (1)_Lect(2)_2022.pdf)
38. [http://www.ee.ic.ac.uk/pcheung/teaching/DE2_EE/Lecture 16 - PID controller \(notes\).pdf](http://www.ee.ic.ac.uk/pcheung/teaching/DE2_EE/Lecture 16 - PID controller (notes).pdf)
39. <https://www.perplexity.ai/search/47c5abb9-a463-43b1-82ce-255a1b04035a>
40. <https://www.perplexity.ai/search/2846ac4a-bc9b-41e1-929d-4afb4e5c4b6f>
41. <https://www.perplexity.ai/search/02e28d96-6967-4de0-91ea-daa936a2b57>
42. <https://www.perplexity.ai/search/eed6da50-25d9-454c-b836-d4068efd7d44>
43. <https://www.perplexity.ai/search/9481c056-ba07-4402-ad09-6d23ec672738>
44. <https://www.perplexity.ai/search/1660ad38-f74a-4080-8032-6ab85e8cc0a5>
45. <https://www.perplexity.ai/search/8f42ca56-a113-4476-a36b-c61624e11259>
46. <https://www.perplexity.ai/search/ccb14842-6c4b-421c-b185-381e13e53b24>
47. <https://www.perplexity.ai/search/762bc4e3-b85f-448a-8679-83c1089d3116>
48. <https://www.semanticscholar.org/paper/Task-Oriented-Co-Design-of-Communication,-and-for-Diao-Zhang/44ea09ba5b995ea22f152ef6eb413600b69784ec>
49. https://research.manchester.ac.uk/files/361287122/2024_JSAC_SI_on_C3ICPS_University_Backup_.pdf
50. <https://www.gla.ac.uk/schools/engineering/staff/index.html/staffcontact/person/4edcece0809e>